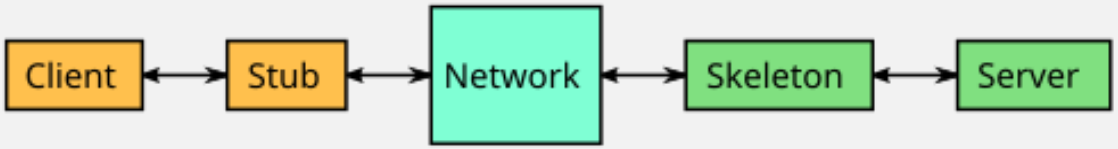
RPC/RMI • HTTP • RABBITMQ/KAFKA • PATTERNS

RPC/RMI • HTTP • RabbitMQ/Kafka • Patterns

SYNCHRONOUS CALL

REMOTE PROCEDURE CALL (RPC)

- History
  - 1980s–1990s: RPC (SunRPC, ONC RPC), CORBA, DCOM
  - Java RMI (1997): remote object calls via stubs/skeletons (**proxy**), Registry
  - Goal: hide the network, call as if local
- Principles
  - Remote interfaces, network exceptions
  - Object marshalling/unmarshalling
  - Tight coupling between client/server types



RPC/RMI – LIMITATIONS

- Deceptive transparency (a remote call is not local)
- Limited scalability (server state, blocking, timeouts)
- Latency and retry policy often on the client
- Restricted interoperability (language/platform)
- Resilience: hard to handle partial failures and backpressure

HTTP REQUEST

HTTPCLIENT

Java 11+ HttpClient: standard API, async via CompletableFuture

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest req = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/users/42"))
    .header("Accept", "application/json")
    .build();
HttpResponse<String> res = client.send(req,
    HttpResponse.BodyHandlers.ofString());
System.out.println(res.statusCode());
System.out.println(res.body());
```

SPRING RESTTEMPLATE (LEGACY)

Blocking, for simple cases

```
RestTemplate rt = new RestTemplate();

ResponseEntity<User> res =
    rt.getForEntity("https://api.example.com/users/{id}", User.class, 42);
if (res.getStatusCode().is2xxSuccessful()) {
    User u = res.getBody();
}
```

SPRING WEBCLIENT (GET ASYNC)

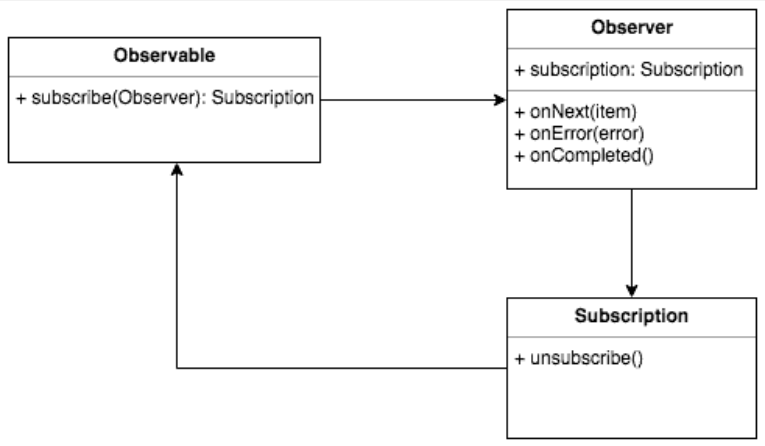
```
WebClient webClient = WebClient.builder()
    .baseUrl("https://api.example.com")
    .defaultHeader(HttpHeaders.ACCEPT, MediaType.APPLICATION_JSON_VALUE)
    .build();

Mono<User> mono = webClient.get()
    .uri("/users/{id}", 42)
    .retrieve()
    .bodyToMono(User.class);

mono.subscribe(u -> System.out.println(u.getName()));
```

SPRING WEBCLIENT

- Leverages **Reactive Programming** principles



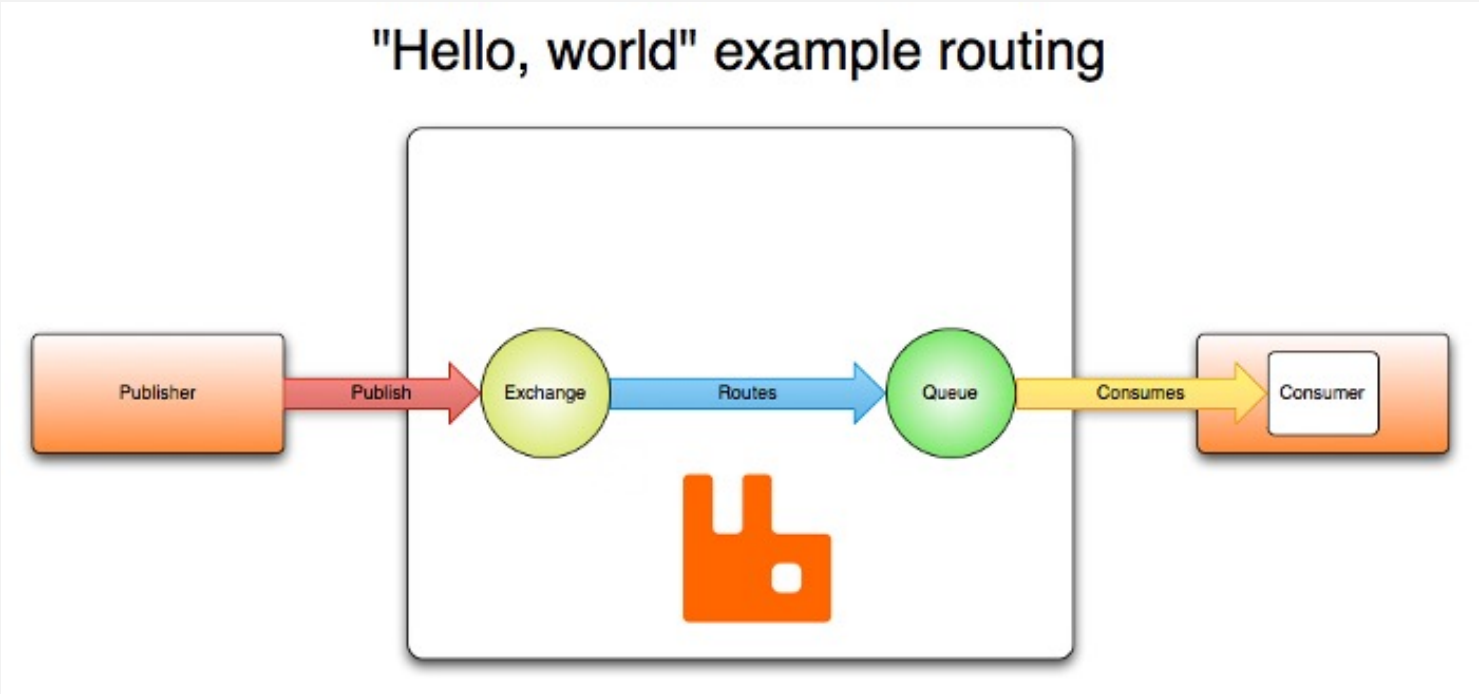
- Backpressure management with **WebFlux**

MESSAGING

MESSAGING – RABBITMQ

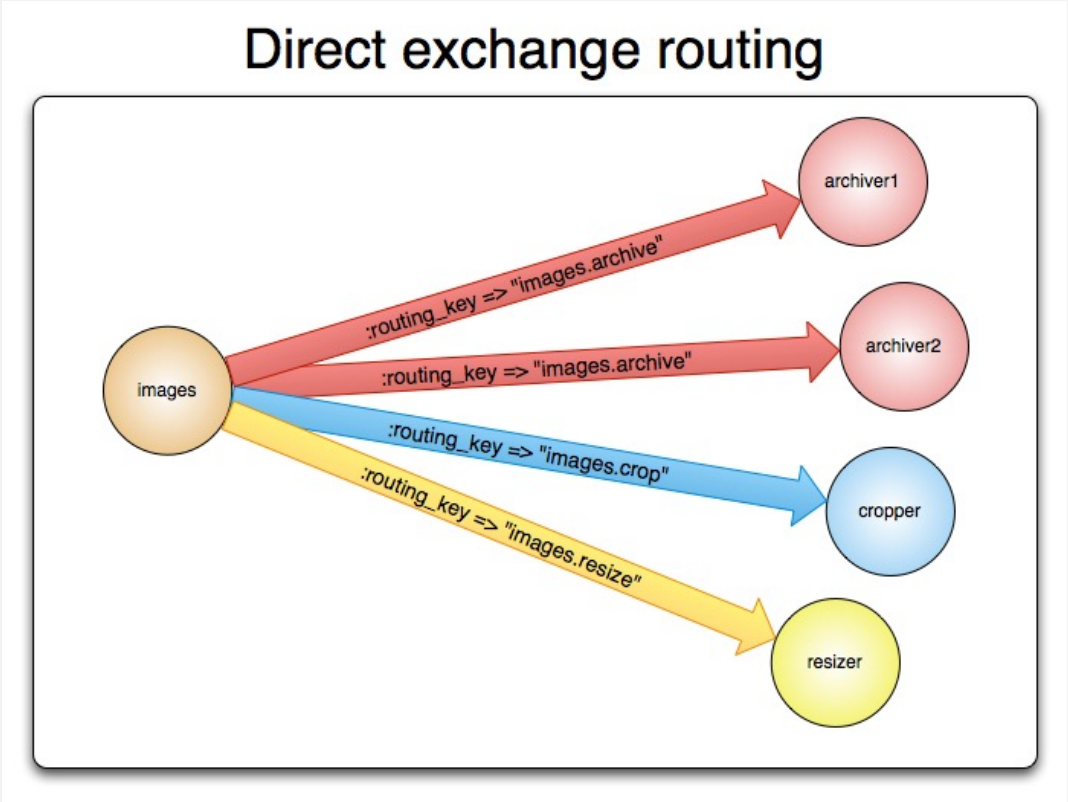
- Advanced Message Queuing Protocol (AMQP)
- Producer → Exchange → Queue → Consumer
- Temporal decoupling (async), buffers, retries
- Contracts (JSON/Avro) + headers (correlationId, schema-version)
- Dead Letter Queue (DLQ) to isolate failed messages

EXCHANGE


"Hello, world" example routing
Publisher
Publish
Exchange
Routes
Queue
Consumes
Consumer

DIRECT EXCHANGE (RABBITMQ)

- Queues bind with a routing key K
- A message arrives with routing key R
- Exchange routes to all queues where K = R



EXCHANGES

- **Direct**: one exact key (e.g., home.kitchen.temperature)
- **Default**: each queue is bound to its own name
- **Fanout**: message is delivered to every bound queue
- **Topic**: pattern keys (e.g., home.*.temperature)
- **Headers**: routing on attributes

SPRING RABBITTEMPLATE (PUBLISHER)

```
@Service
public class Publisher {

    private final RabbitTemplate rabbit;

    public Publisher(RabbitTemplate rabbit){ this.rabbit = rabbit; }

    public void sendDeliveryProcccessed(Map<String,Object> evt){
        rabbit.convertAndSend(”delivery",
            ”order.confirm", evt);
    }
}
```

Injection

SPRING @RABBITLISTENER (CONSUMER)

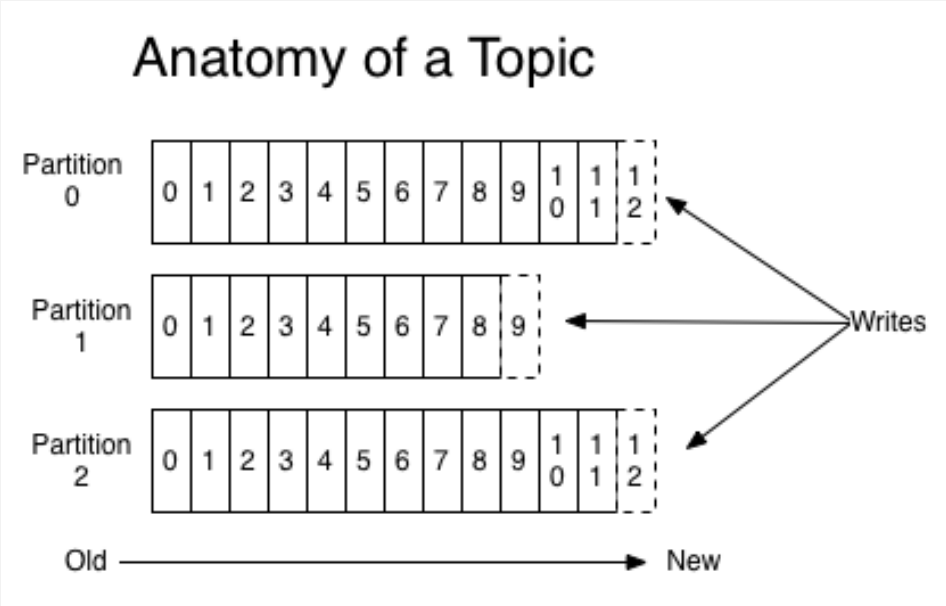
```
@Component
public class Listener {
    @RabbitListener(queues = "order.request")
    public void onMessage(Map<String,Object> evt){
        System.out.println("Event: " + evt);
    }
}
```

https://www.baeldung.com/rabbitmq-spring-amqp

KAFKA

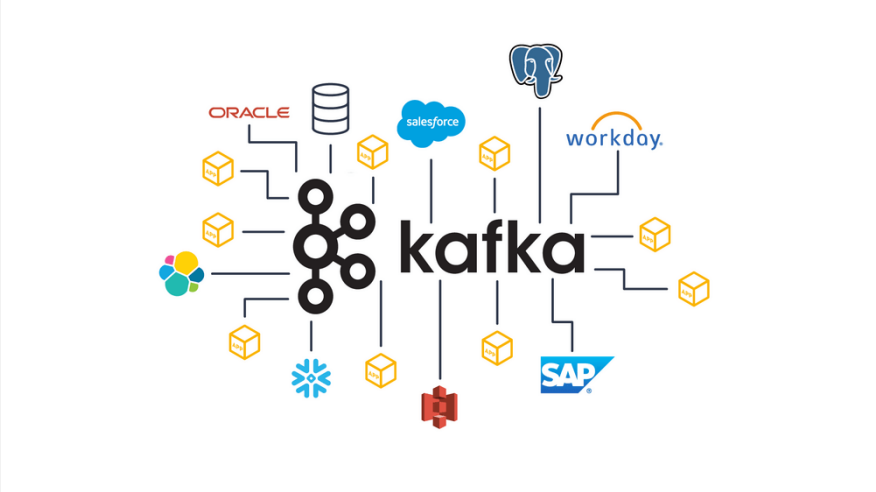
KAFKA – KEY CONCEPTS

- Distributed log: Topics → Partitions → Offsets
- Producers write, consumers read ahead
- Consumer groups: parallelism and partition assignment



KAFKA – WHEN & HOW

- High-throughput streaming/events, configurable retention
- Replay/Time-travel (offset), exactly-once via idempotence
- Integration: Kafka Streams, ksqlDB, CDC connectors
- **spring-kafka**



USE CASES

USE CASES

- E-commerce: OrderCreated → Stock, Billing, Email (topic)
- IoT Telemetry: Sensors → Aggregation → Alerting (fanout)
- Billing: PDF & Archiving – InvoiceReady → PDF → Archive (direct)
- **Authentication :** registering (email), analytics & verification, fraud detection, token invalidation, …

ARCHITECTURE A – BROKER-CENTRIC (RABBITMQ)

- Microservices → topic exchange → dedicated queues
- Per-queue DLQ, retry via requeue
- Good for async workflows and notifications

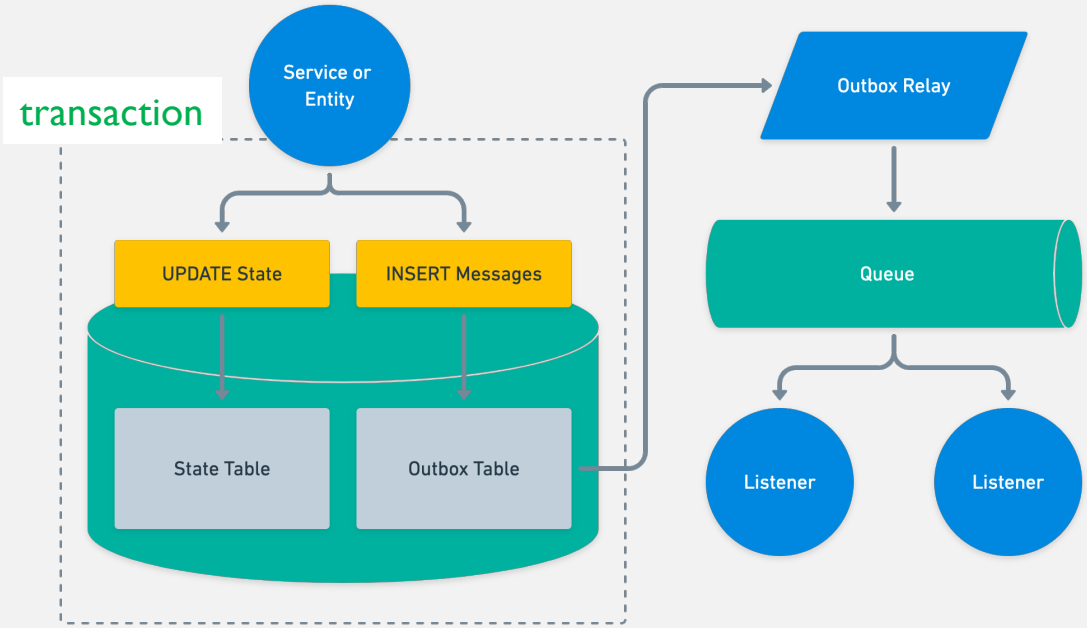
ARCHITECTURE B – STREAMING (KAFKA)

- Producers → partitioned topic → processors (Streams) → sinks
- State stores, joins, replay, time windows
- Great for realtime analytics and event pipelines

PATTERN

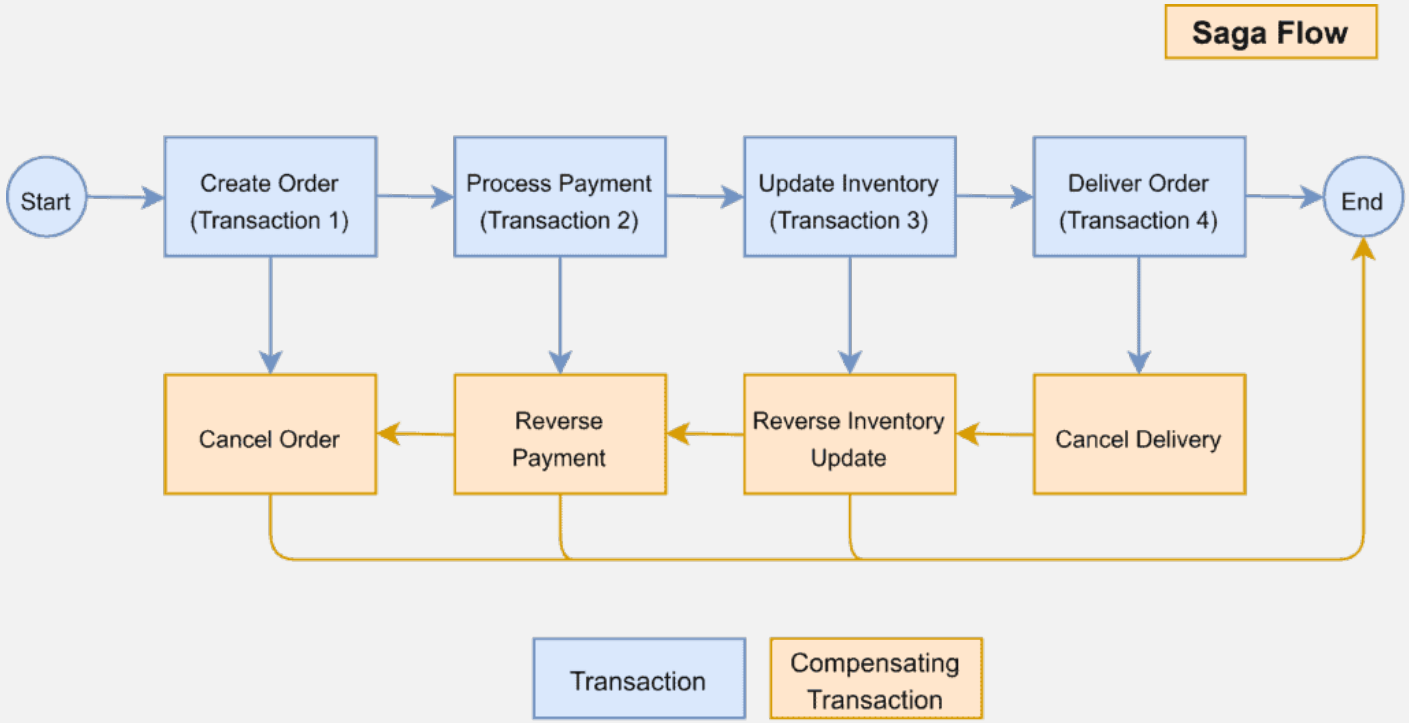
PATTERN – OUTBOX

- BD:Write (data + event) in the same transaction
- Relay reads outbox table, publishes to broker
- Prevents event loss / double write
- (DB commit ≈ guaranteed emission)



PATTERN – SAGA (CHOREOGRAPHY)

- Chain of events; each service reacts and emits
- Compensations if a step fails (e.g., refund)
- Alternative: dedicated orchestrator



PATTERN – IDEMPOTENCY

- Consumer stores processed eventId/jti (TTL)
- Replaying must not duplicate effects
- Use business keys (e.g., jti) + unique constraints

PATTERN – RETRY/BACKOFF & DLQ

- Exponential retry, capped
- Route to DLQ for manual/automatic analysis
- Redrive after fix (poison messages)

PATTERN – SCHEMA & COMPATIBILITY

- Evolve messages without breaking (additive)
- JSON Schema/Avro + registry, versioning
- Backward/forward/full compatibility

PATTERN – CORRELATION & TRACING

- Headers: correlationId, causationId
- End-to-end propagation (logs, traces)
- Helps debug and audit flows

CONCLUSION & NEXT STEPS

- Choose synchronous (RPC/HTTP) vs asynchronous (messaging) based on needs
- RabbitMQ: routing, DLQ; Kafka: streaming & replay
- Apply patterns for reliability and scalability